

# SQL Commands Reference

DDL • DML • DQL • DCL • TCL — with practical examples

| Category | Full Form                    | Purpose                             |
|----------|------------------------------|-------------------------------------|
| DDL      | Data Definition Language     | Defines/modifies database structure |
| DML      | Data Manipulation Language   | Manipulates data within tables      |
| DQL      | Data Query Language          | Retrieves data from tables          |
| DCL      | Data Control Language        | Controls access & permissions       |
| TCL      | Transaction Control Language | Manages transactions                |

## 1. DDL — Data Definition Language

Defines and modifies database object structure. Changes are auto-committed (permanent immediately).

### CREATE — Create a new table

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    age INT  
);
```

### ALTER — Add a new column to an existing table

```
ALTER TABLE students  
ADD COLUMN email VARCHAR(100);
```

### DROP — Delete a table entirely

```
DROP TABLE students;
```

### TRUNCATE — Remove all rows, keep structure

```
TRUNCATE TABLE students;
```

### RENAME — Rename a table

```
RENAME TABLE students TO learners;
```

### COMMENT — Add a comment to a table

```
COMMENT ON TABLE students IS 'Stores student records';
```

## 2. DML — Data Manipulation Language

Used to insert, update, or delete data within tables. Changes can be rolled back until committed.

### INSERT — Add a new record

```
INSERT INTO students (id, name, age)
VALUES (1, 'Ravi Kumar', 20);
```

### UPDATE — Modify existing records

```
UPDATE students
SET age = 21
WHERE id = 1;
```

### DELETE — Remove a record

```
DELETE FROM students
WHERE id = 1;
```

### MERGE — Insert or update conditionally (upsert)

```
MERGE INTO students AS target
USING new_students AS source
ON target.id = source.id
WHEN MATCHED THEN
    UPDATE SET target.name = source.name
WHEN NOT MATCHED THEN
    INSERT (id, name, age)
    VALUES (source.id, source.name, source.age);
```

### CALL — Execute a stored procedure

```
CALL update_student_age(1, 22);
```

### LOCK TABLE — Lock a table for concurrency control

```
LOCK TABLE students IN EXCLUSIVE MODE;
```

## 3. DQL — Data Query Language

Used purely to query and retrieve data from one or more tables.

### **SELECT — Retrieve all columns from a table**

```
SELECT * FROM students;
```

### **SELECT (filtered) — Retrieve specific columns with a condition**

```
SELECT name, age  
FROM students  
WHERE age > 18  
ORDER BY age DESC;
```

### **SELECT (JOIN) — Retrieve data across related tables**

```
SELECT s.name, c.course_name  
FROM students s  
JOIN enrollments e ON s.id = e.student_id  
JOIN courses c ON e.course_id = c.id;
```

## 4. DCL — Data Control Language

Controls user access and permissions on database objects.

### **GRANT — Give a user access privileges**

```
GRANT SELECT, INSERT ON students TO 'user1';
```

### **REVOKE — Remove previously granted privileges**

```
REVOKE INSERT ON students FROM 'user1';
```

## 5. TCL — Transaction Control Language

Manages transactions so that DML changes can be made permanent or undone as a group.

### COMMIT — Save all changes permanently

```
BEGIN;  
UPDATE students SET age = 21 WHERE id = 1;  
COMMIT;
```

### ROLLBACK — Undo changes since the last commit

```
BEGIN;  
DELETE FROM students WHERE id = 1;  
ROLLBACK; -- deletion is undone
```

### SAVEPOINT — Set a point within a transaction to roll back to

```
BEGIN;  
UPDATE students SET age = 21 WHERE id = 1;  
SAVEPOINT age_update;  
DELETE FROM students WHERE id = 2;  
ROLLBACK TO age_update; -- only the delete is undone  
COMMIT;
```

### SET TRANSACTION — Set properties for the transaction

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```